

# Closed-Loop Threat Response Simulation

Technical Approach, Prototype and Results

Prepared by Nina Haag · Candidate, R&D Modelling & Simulation Engineer

September 2026

Each technical section starts with a short plain-language explanation, followed by the engineering detail. Sections 1–6 describe the intended design; Section 7 describes the simplified 2D prototype that was actually coded and run, and the results are evidence about that prototype only.

## Note to the reviewers

I’m currently in a full-time role, so I scoped this to a simplified 2D prototype plus a design for the full system. Sections 1–6 describe the design I would build; Section 7 describes the prototype I actually coded, ran and debugged, and says exactly where the two differ. Results are evidence about the prototype only. Snippets in Sections 2–6 are illustrative sketches; the working code accompanies this document.

One thing I want to be upfront about, in the spirit the brief invites: my background is in space hardware R&D — radiation effects, thermal design, sensor and electro-optical systems, qualification. I have not built flight GNC / closed-loop control software before. What I bring to this task is not polished estimator or controller code; it is a grounded sense of where the real hardware will fight the idealised model, which is where I’ve concentrated the analysis below. I would genuinely welcome the chance to walk through any section in more depth on the follow-up call.

## 1 Architecture

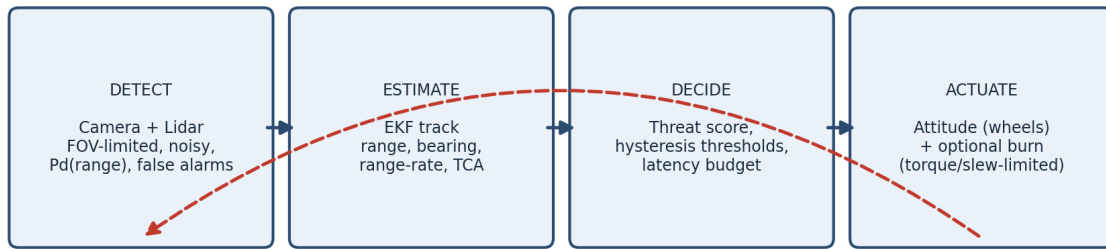
Sections 1–6 describe the intended full design. What was actually coded and run is a simplified 2D prototype; the table in Section 7 lists the differences, and all numerical results refer to the prototype only.

Picture a satellite (the “host”) that needs to protect itself from another spacecraft (the “chaser”) trying to sneak up and dock with it without permission. The host has a camera and a small laser rangefinder to spot the chaser, a piece of software that figures out how dangerous the situation is, a piece of software that decides what to do about it, and thrusters/spinning wheels that physically turn or nudge the satellite out of the way.

The tricky part — and the part this whole assignment is really testing — is that these four steps form a *loop*, not a straight line. If the satellite turns to protect itself, it might turn its camera away from the very thing it’s trying to watch. So the “fix” can accidentally cause a new problem: going blind at the worst possible moment. Good design has to account for that trade-off, not just each step in isolation.

The loop is Detect → Estimate → Decide → Actuate, with one feedback path that the task correctly identifies as the crux of the problem: the host’s own manoeuvre changes its attitude, which changes what the sensor can see next.

### Closed-Loop Threat Response Architecture



feedback: manoeuvre changes host attitude -> changes sensor FOV/visibility -> changes next detection

Two design choices follow directly from this diagram:

- The estimator must run on every sensor tick regardless of whether a manoeuvre is in progress — attitude change mid-manoeuve must feed back into what “currently visible” means, not be treated as a dead interval.
- The decision block must know the sensor’s field-of-view geometry, not just the target state: a manoeuvre that turns the docking port away from the chaser also turns the camera (they are fixed to the same body), and once the chaser is more than about 30° off the boresight (half the 60° field of view) it is no longer observable, so the host loses the very track that told it to manoeuvre.

## 2 Sensor model — where I’ve concentrated the effort

This section is about the satellite’s “eyes”: a camera and a laser rangefinder (lidar). Real cameras and lidars don’t work perfectly — they sometimes fail to spot something that’s really there (a *missed detection*), sometimes think they see something that isn’t there (a *false alarm*), and they get worse at spotting things the further away those things are. This section is where I have the most direct hands-on knowledge, because it’s closest to my hardware background, so I’ve gone into more depth here than anywhere else in the document.

### 2.1 Detection probability vs. range

Think of “detection probability” as: if the chaser is at a certain distance, what is the chance the camera actually notices it on any given check? Close up the chance is high (95%); far away it is lower (40% at 800 metres). Instead of drawing an arbitrary line between those two numbers I used a smooth S-shaped curve, a common empirical choice: detection tends to fade gradually rather than switch off at one exact distance as the signal weakens relative to the sensor’s noise.

The brief allows a self-defined detection-probability curve. I use a logistic curve parameterised by a 50%-detection range and a transition width. It is an *empirical surrogate* for the fall-off of detection with range, not a first-principles optical model: real detection depends on background, integration time, the point-spread function, thresholding and target reflectance, and range scaling of signal

(roughly  $1/R^2$  for passive optics, roughly  $1/R^4$  for the lidar return under simple assumptions) is not the same thing as a detection-probability curve. The two parameters are solved exactly from the brief's anchor points ( $P_d(200) = 0.95$ ,  $P_d(800) = 0.40$ ), giving  $r_{50} = 727.4$  m and  $k = 0.00558 \text{ m}^{-1}$  (a self-check in the code confirms the curve reproduces both anchor points).

```
def pd_curve(range_m, r50=727.4, k=0.00558):
    """Logistic detection probability (empirical surrogate).
    r50: range at which Pd=0.5 (larger = better sensor).
    Solved from Pd(200)=0.95 and Pd(800)=0.40:
    k = [logit(0.40)-logit(0.95)]/600, r50 = 800 - logit(0.40)/k"""
    return 1.0 / (1.0 + np.exp(k * (range_m - r50)))
```

## 2.2 What I would NOT leave as a static curve

A single curve that only depends on distance is a fair simplification for a one-off test, but it hides three real-world effects that any actual satellite camera would run into. I list them here on purpose, even though I haven't built them into the model, because pretending they don't exist would be dishonest about how good this simulation really is:

- **Glare from the sun:** if the chaser approaches from a direction close to where the sun is, the camera can get "blinded" by bright light regardless of how close or far the chaser actually is — similar to how it's hard to see a car coming toward you when the sun is right behind it.
- **Slow wear from radiation:** space is full of radiation, and over months or years it damages the camera's detector, making it noisier and less sensitive. This wouldn't matter for one 30-minute test, but it would matter over the years-long life of a real mission.
- **Motion blur when turning:** if the satellite is actively rotating to defend itself, its own camera image blurs a bit, just like a photo taken from a moving car. That temporarily makes it worse at spotting the target, right when it needs to see clearly the most.

A pure range-based detection curve is the honest simplification for the assignment, but it hides three effects that I flag as known gaps, because they are the kind of thing that bites in reality:

- **Solar/Earth-limb glare:** a chaser approaching from certain V-bar/R-bar geometries can sit close to the sun vector in the host's field of view. Real electro-optical detectors saturate or bloom in that geometry regardless of range. I'd model this as a geometry-dependent detection penalty, not just a range one, if time allowed.
- **Radiation-driven dark current growth:** total ionizing dose and displacement damage raise a CMOS/CCD detector's dark current over mission life, which raises the noise floor and quietly erodes the effective detection curve over years — not something a single-scenario Monte Carlo would ever surface, but worth naming as a limitation of treating detection probability as time-invariant.
- **Slew-induced blur/blinding:** the brief flags that a slew can blind the sensor — I'd also model short-duration bearing-noise inflation during active slewing (motion blur increases effective angular noise), not just a hard field-of-view cutoff, since real detectors don't have a clean on/off boundary.

## 2.3 Measurement generation with dropouts, misses and false alarms

This code is the “sensor simulator.” Every time the camera takes a picture, this logic decides: was the target even in view? If yes, did the camera actually spot it (based on the fading probability curve above)? If it did spot it, the reading isn’t perfectly accurate — it comes with a small random error, just like a real measurement would. Separately, every so often the camera might report something that isn’t really there at all (a false alarm), which the decision-making system downstream has to be smart enough not to overreact to.

```
# Illustrative sketch; the working version is sensor_tick() in simulate.py
def simulate_camera_frame(true_bearing, true_range, in_fov, rng):
    if not in_fov:
        return None # no access, not just "no detection"
    pd = pd_curve(true_range)
    if rng.random() > pd:
        return None # missed detection
    bearing_meas = true_bearing + rng.normal(0, np.deg2rad(0.1))
    return {'type': 'detection', 'bearing': bearing_meas}

def maybe_false_alarm(rng, p_fa=0.02):
    if rng.random() < p_fa:
        # false alarm: random bearing within FOV, no lidar correlate
        return {'type': 'false_alarm',
                'bearing': rng.uniform(-np.deg2rad(30), np.deg2rad(30))}
    return None
```

The brief says the lidar is valid only inside the camera field of view. In this design I go one step further, as a simplifying modelling assumption: the lidar range is used only when the camera also detects the target (the camera detection supplies the association). This is stronger than the brief requires; a lidar could in principle return a range without a camera detection.

## 3 Relative dynamics

This section is about the physics of two objects moving near each other in orbit — not floating in empty space, but both circling the Earth together, which makes their relative motion behave in a specific, well-studied way. Rather than simulating both full orbits, a standard set of equations, the Clohessy-Wiltshire equations, describes how nearby objects move relative to each other under simplifying assumptions (near-circular orbit, small separation). It is a standard approximation for short-range problems like this one, and its limits are stated below.

Clohessy-Wiltshire (CW) is an appropriate and standard tool here, and the brief explicitly allows it. It assumes a circular reference orbit and a separation small compared with the orbital radius. The separation condition holds easily for 300–800 m at 400 km altitude. The time span is less comfortable: 30 minutes is about a third of the 92.6-minute orbital period, and the neglected nonlinear terms accumulate, so for a real study I would compare CW against a nonlinear two-body propagation over the full scenario time and report the position error. I did not do that comparison.

```
def cw_matrix(n): # 3D form; the prototype uses the planar 4x4 version
    """Continuous-time CW state matrix for state [x,y,z,vx,vy,vz],
    n = orbital mean motion (rad/s)."""
    A = np.zeros((6, 6))
    A[0:3, 3:6] = np.eye(3)
```

```

A[3, 0] = 3 * n**2; A[3, 4] = 2 * n
A[4, 3] = -2 * n
A[5, 2] = -n**2
return A

# n for 400 km circular LEO:
mu = 3.986004418e14
a = 6378137 + 400e3
n = np.sqrt(mu / a**3) # ~0.00113 rad/s, period ~92.6 min

```

The propagator is integrated with a fourth-order Runge-Kutta scheme at a 0.5 s step and compared with the closed-form CW solution for a purely radial offset, which is part of self-check #3 in the brief (the error is below  $10^{-4}$  m over 500 s). Note that in the Monte Carlo runs of the prototype the chaser is moved by a commanded velocity, not by this propagator (Section 7).

## 4 Detection & estimation

The sensor does not hand over a clean answer like “the chaser is at exactly this position.” It hands over noisy, occasional readings, and the software has to combine them into a smooth best guess of where the chaser is and how fast it is closing. This process is called *estimation*. The design uses a well-known technique, an Extended Kalman Filter; the coded prototype uses a simpler relative, an alpha-beta filter (see below). Think of either as a smart averaging machine that also predicts forward in time and corrects itself as new, imperfect readings arrive. Just as important: because false alarms exist, the system shouldn’t instantly believe the very first blip it sees. It should wait for a short pattern of consistent readings before saying “yes, this is really a target,” the same way you wouldn’t slam the brakes on a highway because you glimpsed something out of the corner of your eye for a split second.

I’d use an Extended Kalman Filter (EKF) over a plain Kalman filter, and here’s the actual justification rather than a default choice: the dynamics (Clohessy-Wiltshire) are linear, but the measurements are not — bearing is an angle computed from x/y position in the sensor frame, which is a nonlinear function of the state. An EKF linearises that measurement relationship at each update; a plain Kalman filter would require converting bearing and range to straight-line (Cartesian) coordinates first, which just moves the nonlinearity into a separate triangulation step without removing it. I’d rather keep one filter and one clearly-stated linearisation point than two stages each hiding an approximation.

Track initiation matters a great deal here, given a 2% per-frame false-alarm rate. A single detection should never start a confirmed track — I use “M-of-N” logic (e.g. 3 plausible, spatially-consistent detections out of 5 consecutive frames) before the filter is initialised, otherwise false alarms produce phantom tracks. The snippet is an illustrative sketch of the prototype’s gate.

```

# Illustrative sketch; see TrackInitiator in simulate.py
class TrackInitiator:
    def __init__(self, m=3, n=5, gate_deg=2.0):
        self.buffer = [] # recent (frame_idx, bearing) or None
        self.m, self.n, self.gate = m, n, np.deg2rad(gate_deg)

    def update(self, meas):
        self.buffer = (self.buffer + [meas])[-self.n:]
        plausible = [b for b in self.buffer if b is not None]
        if len(plausible) < self.m:
            return False
        bearings = [b['bearing'] for b in plausible]

```

```
return (max(bearings) - min(bearings)) < self.gate
```

The design calls for reporting the track initiation range (the true range at the moment the track is created, which differs from the range of the first raw detection) and the estimation error against range. The prototype reports the track initiation range and per-run RMS estimation errors, but not the error binned by range (Section 7).

**What the prototype actually does.** The prototype uses an alpha-beta filter, not an EKF: it has no covariance and no innovation gating, so the EKF discussion above is the design rationale, not something that was run. The filter gains were originally too high for the sensor noise the model defines and made the range-rate estimate mostly noise; they now follow the critically-damped rule  $\beta = \alpha^2 / (2 - \alpha)$  with  $\alpha = 0.05$  (Section 7.3). The track-initiation test shown above is a toy gate (a fixed  $2^\circ$  bearing spread); a moving target should be gated against a predicted measurement, using the innovation and its covariance.

## 5 Decision logic

Once the system has a decent idea of where the chaser is, something has to decide: is this actually dangerous enough to react to, right now? This is like a smoke alarm — you want it to go off when there’s a real fire, but not every time someone makes toast. The system combines a few clues (is the chaser getting closer? how soon would it arrive? how confident are we that this is a real, tracked object and not noise?) into a single “threat score.”

To avoid the alarm flickering on and off right at the boundary (imagine a thermostat that clicks on and off every second because the temperature is right at the exact setpoint), there are two separate thresholds: one to trigger the response, and a lower one that has to be crossed before standing down again. This gap is called *hysteresis*, and it’s a standard trick to prevent jittery, flip-flopping behaviour.

There’s also a deliberate, honest delay built in between “the system decides something is wrong” and “the satellite actually starts moving.” Real computers and real command chains are not instant — and if this delay is ignored, the whole simulation would be quietly cheating by giving the satellite reflexes no real hardware has.

The threat score is a heuristic: a weighted combination of closing speed, urgency (time-to-closest-approach) and a confidence term. Its limitations matter: “confidence” here is just track age (a filter-covariance measure would be better), time-to-closest-approach says nothing about whether the chaser will actually reach the docking port, and a defensible metric would be the predicted miss distance to the docking corridor with its uncertainty. The urgency term uses a 300 s horizon, chosen so it responds meaningfully over the last several minutes of a typical approach rather than only in the final seconds.

```
def threat_score(track, urgency_horizon=300.0):
    closing = min(max(0.0, -track.range_rate), 1.0) # m/s, capped at 1
    tca = track.time_to_closest_approach() # None if range is opening
    urgency = 1.0 / (1.0 + tca / urgency_horizon) if tca is not None else 0.0
    confidence = min(track.age_frames / 5.0, 1.0)
    return confidence * (0.5 * closing + 0.5 * urgency)

MIN_TRACK_AGE = 25 # frames (5 s): a track must mature before it may arm
ARM_THRESHOLD = 0.6 # two thresholds, to avoid chattering
CLEAR_THRESHOLD = 0.35 # must drop below this to stand down
```

Decision latency is modelled explicitly as a fixed delay (not zero) between “threat score crosses

the arm threshold” and “attitude command issued”, representing onboard compute time for the estimation and decision pipeline. The brief’s self-check #4 says that adding 5 seconds of delay should lower the denial rate, and that if it does not, the loop is not actually closed. In the prototype this check *failed* (Section 7); I report that as a weakness of the prototype rather than treating the latency parameter as validated.

## 6 Actuation

This is the “muscles” of the satellite — how it actually moves once a decision has been made. Satellites usually turn themselves using spinning wheels inside the spacecraft (reaction wheels): speed a wheel up in one direction, and by simple physics (conservation of angular momentum, the same idea behind a spinning ice skater), the rest of the satellite gently rotates the other way. There are hard limits here that a real design has to respect: the wheels can only push so hard, and the satellite can only turn so fast — these aren’t just numbers on a spec sheet, they’re physical ceilings the simulation has to actually enforce moment by moment, not just check at the end.

I’ve also flagged two very real hardware quirks that a purely theoretical simulation would likely miss: wheels can only “store” so much spin before they run out of room and need to be reset (like a jar that eventually fills up and needs emptying), and the small gas thrusters used for larger movements can’t fire in arbitrarily tiny amounts — they have a minimum “puff” size, below which they either don’t fire at all or fire more than intended.

The design uses quaternion kinematics with Euler’s rigid-body equations. The commanded body torque is limited (0.01 N·m per axis) and the body rate is limited (2°/s); both limits are applied at every integration step, not after the fact, so that the controller sees the saturation. With a reaction-wheel array the body torque equals minus the rate of change of the wheels’ angular momentum, so each wheel’s torque and momentum limits and the array geometry set the torque actually achievable; the sketch below treats the actuator as an ideal torque source and does not model this.

```
# Design sketch (3-axis body torque); the prototype is single-axis, see Section 7
def attitude_step(q, w, torque_cmd, inertia, dt, max_torque=0.01):
    torque = np.clip(torque_cmd, -max_torque, max_torque)
    wdot = np.linalg.solve(inertia, torque - np.cross(w, inertia @ w))
    w_new = w + wdot * dt
    # slew-rate saturation, per-axis
    w_new = np.clip(w_new, -np.deg2rad(2), np.deg2rad(2))
    q_new = quat_integrate(q, w_new, dt)
    return q_new, w_new
```

Two hardware-reality details I’d insist on including, because they’re exactly the kind of thing a controls-focused simulation tends to idealise away:

- **Wheel momentum accumulation:** a sustained torque of 0.01 N·m over 1800 s corresponds to 18 N·m·s of accumulated angular momentum. Whether this saturates the wheels depends on the momentum capacity of the wheel actually selected, which I did not check for a specific unit. In a real design I would track cumulative wheel momentum and flag any run that would need desaturation (magnetorquer or thruster). The implemented prototype does *not* model this: its actuator is an ideal torque- and rate-limited single-axis source.
- **Minimum impulse bit quantization:** the brief gives the cold-gas thruster a 0.01 m/s minimum impulse. Commanded delta-v below that threshold either cannot be delivered or is over-delivered, so a design should quantise commanded burns to multiples of it. The prototype does not model host burns.

## 7 Monte Carlo & sensitivity sweep — implemented, and run

No single test run tells you much, because the chaser’s starting position and the sensor noise are random each time. The scenario is therefore repeated 200 times with different random conditions, and the results are turned into an overall “denial rate” with a confidence interval (a range showing how much the number could reasonably vary if the experiment were repeated). A “sensitivity sweep” then changes one setting at a time to see which settings move the outcome.

This section describes a simplified 2D prototype that was actually coded, run and debugged. It is not the full 3D design of Sections 1–6; the next subsection says exactly where they differ.

### 7.1 What was designed versus what was implemented

| Part            | Designed (Sections 1–6)                         | Implemented in the code                                                                                                                                                                                                                                                                                                                                                  |
|-----------------|-------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Relative motion | 3D Clohessy-Wiltshire (CW)                      | A planar CW propagator is implemented and checked against the closed-form solution, but it is <i>not</i> used in the Monte Carlo runs. There, the chaser follows a commanded straight-line velocity toward an aim point on the host’s port axis, with braking inside 60 m; it is effectively assumed to cancel orbital drift continuously. The host stays at the origin. |
| Chaser budget   | 1.0 m/s delta-v (brief)                         | Only re-aiming is charged: cost = closing speed $\times$ the change in the unit heading vector. Closing and braking burns are treated as free.                                                                                                                                                                                                                           |
| Estimator       | Extended Kalman filter                          | Alpha-beta filter on range and bearing, $\alpha = 0.05$ , $\beta = \alpha^2/(2 - \alpha)$ ; no covariance, no statistical gating                                                                                                                                                                                                                                         |
| Attitude        | Quaternion model, 3-axis, reaction-wheel array  | One body angle; an ideal actuator limited to 0.01 N·m about an axis of inertia 2.0 kg·m <sup>2</sup> (one of the brief’s principal moments; i.e. 0.005 rad/s <sup>2</sup> ) and 2°/s; no quaternions, no wheel-momentum model                                                                                                                                            |
| Camera and port | Separate body axes (+X and +Z)                  | The <i>same</i> axis (one angle)                                                                                                                                                                                                                                                                                                                                         |
| Sensors         | Camera 5 Hz and lidar 2 Hz                      | Camera bearing and lidar range are both sampled at 5 Hz (the lidar is not rate-limited to 2 Hz); range is only used when the camera also detects; logistic $P_d(r)$                                                                                                                                                                                                      |
| Host thrusters  | Minimum-impulse-bit burns, 0.5 m/s budget       | Not modelled; the host does not translate                                                                                                                                                                                                                                                                                                                                |
| Threat metric   | Predicted miss distance to the docking corridor | Heuristic score from closing speed, time-to-closest-approach and track maturity                                                                                                                                                                                                                                                                                          |

Results below are evidence about the implemented prototype only, not about the full design. Planar motion is justified by the brief’s own statement that it is acceptable: in the linearised CW equations the cross-track motion is decoupled from the in-plane motion, and the perception–action coupling under study (field of view versus attitude) is present in the plane. The brief prefers 3D; this was not done.

### 7.2 What “denied” means

A run is **docked** if, at some time step, the chaser is within 5 m of the host, the direction from host to chaser is within 15° of the host’s port axis, and the chaser’s speed is at most 0.1 m/s. Every other run is **denied**, following the brief: either the chaser exhausts its 1.0 m/s delta-v budget (**denied\_dv\_exhausted**) or 30 minutes elapse first (**timeout**). The host never moves translationally, so “denial” means that the host has rotated its port axis so that this particular chaser cannot complete docking — not that it has flown out of the way. The rate is therefore a property of the host design, the chaser’s fixed guidance rule, the chaser’s budget accounting and the scenario

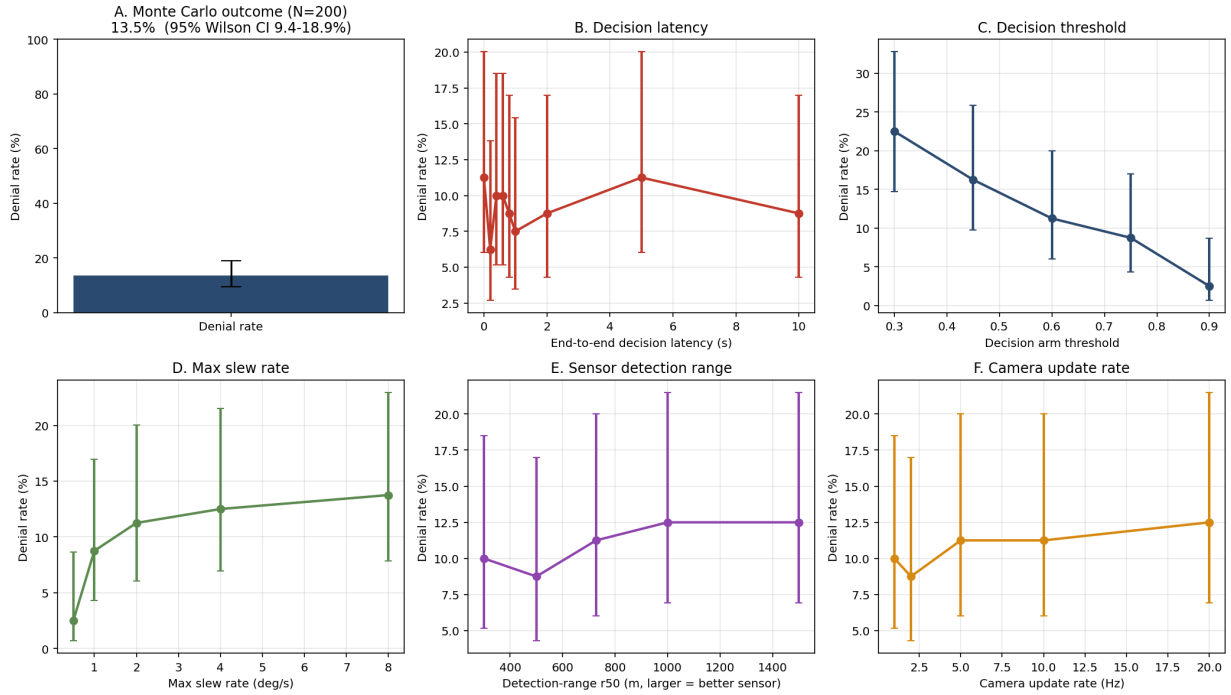
distribution together, not of the host alone. Scenarios are drawn with the pure closing time limited to 1500 s (otherwise some draws cannot dock within 30 minutes regardless of the host), which narrows the range tested.

### 7.3 Calibration and modelling notes

A few numbers in this model needed careful calibration to actually mean what they claim to mean. The two worth flagging: the sensor's detection-probability curve, and the filter that turns noisy readings into a speed estimate.

The detection-probability curve is solved exactly from its two anchor points (95% detection at 200 m, 40% at 800 m), giving  $r_{50} = 727.4$  m and  $k = 0.00558 \text{ m}^{-1}$ ; a self-check now confirms the curve actually hits both points. The range/bearing filter uses gains tuned so its speed (range-rate) estimate stays well below the true closing speeds being measured (error about 0.03–0.12 m/s against closing speeds of 0.3–1 m/s); with looser gains, the filter's own noise was large enough to occasionally exceed the arming threshold on its own, which would have made the decision logic react to noise rather than to the chaser. A track must also be at least 5 s old before it is allowed to trigger arming, so a single noisy reading cannot do so. Two bookkeeping metrics (a field-of-view-loss counter and the track-initiation range) were corrected to measure what their names say; neither correction changes the denial rate reported below.

### 7.4 Results



Panel A is the headline: in 200 randomised attempts the host denied docking 27 times. Panels B–F change one setting at a time. The error bars show how uncertain each point is — but the fairest way to decide whether a setting matters is to re-run the *same* scenarios under two settings and count how many outcomes flip, which is what the paired-comparison table below does.

**Headline (N=200):** 27 of 200 runs denied, i.e. 13.5% (95% Wilson interval 9.4–18.9%). Of the 27

denials, 24 were `denied.dv_exhausted` and 3 were `timeout`; because the re-aim cost model is my own simplification (Section 7 table), the rate depends on it.

**Metrics requested by the brief** (baseline Monte Carlo, N=200):

| Metric                                                                    | Result                                                                                                                                                   |
|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Denial rate                                                               | 13.5% (95% Wilson interval 9.4–18.9%)                                                                                                                    |
| Track initiation range (true range at track creation)                     | Median 3 m (interquartile range 2–9 m); a track was confirmed in 197 of 200 runs. In most runs the chaser is first confirmed only in the last few metres |
| Estimation error after a track is 5 s old (per-run RMS, median over runs) | Range 0.22 m; range-rate 0.077 m/s. Not broken down by range                                                                                             |
| False-alarm-triggered manoeuvres per run                                  | 0.000                                                                                                                                                    |
| Delta-v used                                                              | Chaser (re-aiming only): mean 0.165 m/s, maximum 1.0 m/s. Host: none (not modelled)                                                                      |
| Wheel momentum                                                            | Not modelled. Cumulative torque impulse, an upper bound on wheel momentum use: mean 1.65, maximum 13.8 N·m·s                                             |
| Sensor-loss episodes caused by own manoeuvre                              | 0.20 per run on average; every one of the 38 runs in which the host armed lost the chaser from the field of view at least once                           |
| End-to-end latency, first detection to commanded attitude reached         | Not measured; the decision latency is a set parameter                                                                                                    |

**Why the number is what it is.** In the baseline run only 38 of 200 scenarios ever led the host to arm (median true range at arming roughly 500–600 m). A track can form early only if the chaser starts inside the fixed 60° field of view, roughly one start in six; in the other 162 scenarios the host never armed at all. Of the 38 armed runs, 25 were denied; 25 of the 27 denials came from this group. The denial rate is therefore limited by *sensing geometry*: whether the chaser is seen early enough. The closed loop is present: in all 38 armed runs, the host’s own slew removed the chaser from the field of view at least once.

The next table compares two settings of one parameter at a time, for example a low versus a high slew rate. Instead of running two separate, independent batches of random scenarios (which makes it hard to tell a real effect from random luck), the *same* 100 scenarios are re-run once under each setting, and we count how many scenarios changed outcome (“flipped”). The final column is a *p*-value from a standard statistical test for this kind of paired yes/no data (an exact McNemar test): a small *p*-value (conventionally below 0.05, and here we use a stricter 0.01 since five comparisons are made) means the flips are very unlikely to be due to chance alone.

**Which settings matter.** Paired comparisons (the same 100 scenarios under two extreme settings; exact McNemar test on the scenarios whose outcome flipped):

| Parameter (extreme settings)           | Denied, A vs B | Flipped (A only / B only) | Exact <i>p</i> |
|----------------------------------------|----------------|---------------------------|----------------|
| Decision threshold 0.3 vs 0.9          | 24 vs 2        | 22 / 0                    | < 0.0001       |
| Max slew rate 0.5 vs 8°/s              | 3 vs 18        | 0 / 15                    | 0.0001         |
| Decision latency 0 vs 10 s             | 11 vs 12       | 2 / 3                     | 1.0            |
| Detection range $r_{50}$ 300 vs 1500 m | 12 vs 15       | 1 / 4                     | 0.38           |
| Camera update rate 1 vs 20 Hz          | 14 vs 14       | 6 / 6                     | 1.0            |

**Conclusion on which parameter decides survivability.** In this prototype, the decision threshold (how quickly the host decides the chaser is a threat) and the maximum slew rate (how fast the host can physically turn) both have large, consistent effects: arming earlier, and turning faster, both give the host more angular change before the chaser arrives, which is consistent with the chaser exhausting its manoeuvring budget while trying to re-aim. Between those two, the threshold effect is larger over the ranges I tested (22 versus 15 flipped scenarios), but such comparisons depend on

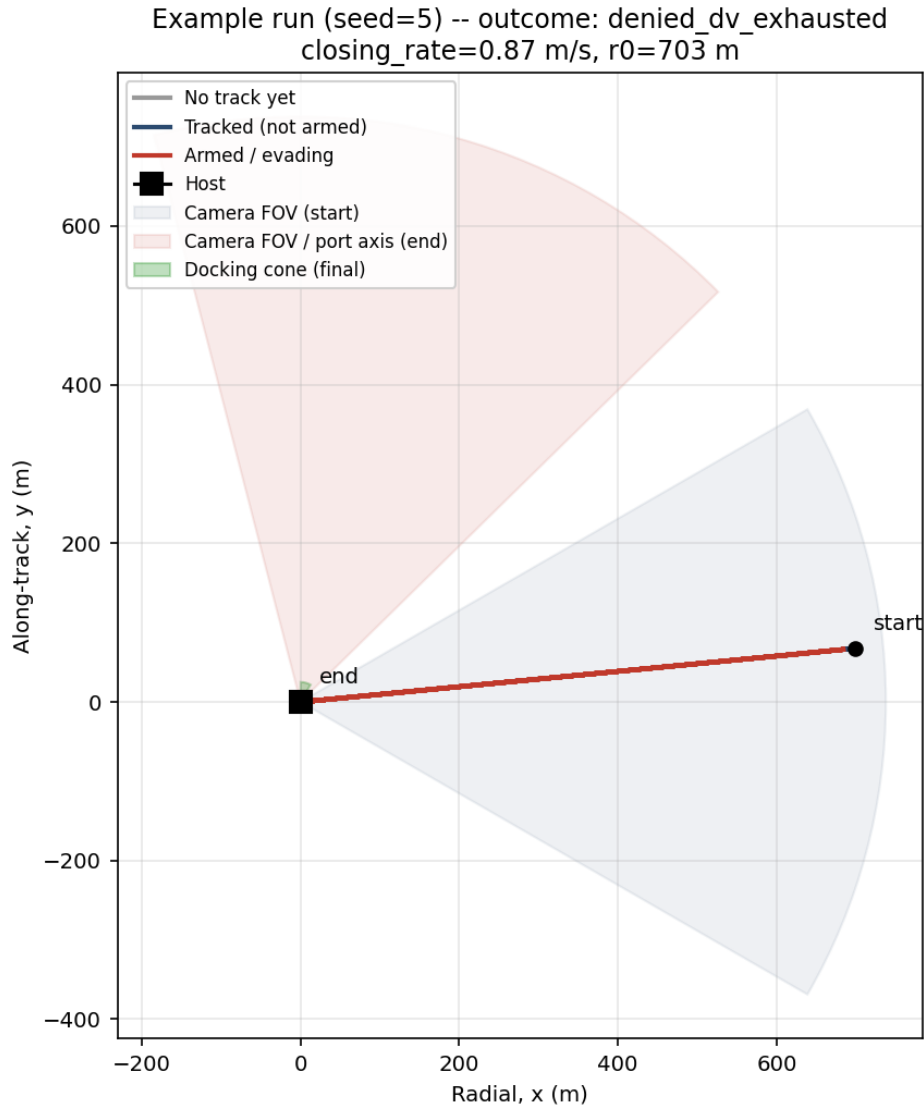
how wide a range is tested, so I do not claim a single dominant parameter. Upstream of both sits the fixed field of view: it decides whether the host ever gets a track early enough for either setting to matter at all. Decision latency, detection range and camera update rate showed no detectable effect: none of the 5 tests below the significance threshold. “No detectable effect” is not the same as proof of no effect — with 100 paired scenarios, only effects of roughly ten percentage points or more would reliably show up. The unpaired error bars in Panels B–F are wide (about  $\pm 5$ –10 points) and overlap, so they should not be used alone to judge whether a setting matters; the paired table above is the more reliable evidence.

## 7.5 Self-checks from the brief

1. **Open-loop sanity: passed.** With an inert host the chaser docked in 100% of 30 runs.
2. **Perfect-sensor bound: passed, with a caveat.** With zero noise, zero false alarms, an effectively infinite detection range and zero latency, the denial rate was 18.5% (200 scenarios), above the 13.5% baseline. It is a ceiling with respect to sensing quality only: other settings can exceed it (for example an arming threshold of 0.3 gave 22.5% in the 80-run sweep), and the 60° field of view was left unchanged.
3. **Dynamics check: partly done.** The planar CW propagation matches the closed-form solution for a purely radial offset (error below  $10^{-4}$  m over 500 s). The brief’s second part, that attitude integration conserves what it should at zero torque, was not run: the prototype has a single angle, for which it is trivial, and no quaternion model.
4. **Latency injection: failed.** The brief expects added decision delay to lower the denial rate, and says that if it does not the loop is not actually closed. In the prototype it did not: 11 versus 12 denials out of 100 paired scenarios for 0 versus 10 s ( $p = 1.0$ ); and with the detection range cut to 150 m or 80 m, so that detection is late, 20 versus 22 and 20 versus 18 out of 200 ( $p = 0.77$  and  $0.75$ ). I do not have a validated explanation. What the data show is that in this prototype the arming decision either happens hundreds of metres out or never, and the outcome is governed by the field of view, the arming threshold and the re-aim cost model, so seconds of delay do not change it. By the brief’s own criterion this is a weakness of the prototype’s coupling, and I report it as one.

An additional check guards the detection curve:  $P_d(200) = 0.950$  and  $P_d(800) = 0.400$ .

## 7.6 One example run



This is a single illustrative run (an off-axis approach from roughly 700 m), useful only as a geometric sanity check: the chaser's path is nearly straight, the host swings its camera and port axis, and the run ends as `denied_dv_exhausted`. It is not statistical evidence; the 200-run result above is the number to use.

## 8 What's wrong with this model

Every simulation simplifies reality. This section lists where the implemented prototype does so, ordered roughly by how much each limitation affects the conclusions. A few of the terms below were introduced earlier in this document (field of view in Section 1, the alpha-beta filter in Section 4, the docking cone and delta-v budget in the scenario description); they are used here without re-explaining them.

- **Camera and docking port share one axis.** This is the largest limitation. It makes the outcome depend on whether the chaser starts inside a fixed 60° field of view (track initiation happens at a median true range of 3 m), and it makes sensing and defending compete for one variable: keeping the chaser in view points the port at it. Decoupling the two axes is the first

extension I would make.

- **The latency-injection self-check failed** (Section 7). The prototype does not reproduce the latency sensitivity the brief expects, so its perception–action coupling is weaker than intended, and the conclusion about which parameter matters is conditional on that.
- **The chaser is kinematic and its budget accounting is my own.** “Kinematic” here means it flies a commanded straight-line path rather than following the orbital equations of motion; only its re-aiming manoeuvres are charged to its 1.0 m/s manoeuvring budget (at closing speed times the change in heading), while closing and braking are treated as free. 24 of the 27 denials were this budget running out, so the denial rate depends directly on this cost model.
- **The threat metric is a heuristic, and the results depend on it.** The threshold sweep shows it is an influential design choice. A defensible metric would instead use the predicted miss distance to the docking corridor together with the estimate’s uncertainty, rather than the closing-speed/time-to-approach heuristic used here.
- **“Denial” depends on the chaser’s policy.** The chaser follows a fixed, pre-programmed approach strategy rather than an adaptive one, and the host never changes its own position, only its orientation. The denial rate is therefore a property of this specific match-up, not a general bound on how hard the chaser would be to stop.
- **Estimator.** The filter that turns noisy readings into a speed estimate (an “alpha-beta filter”) uses fixed, hand-tuned gains rather than the more principled Kalman-filter approach, which would also track its own uncertainty and use that to reject implausible readings. The track-confirmation rule (three detections within a narrow  $2^\circ$  angular spread) is a simple placeholder; a more realistic version would predict where the next reading should fall and check new readings against that prediction and its uncertainty, rather than against a fixed window.
- **Actuator.** The host’s turning is modelled as an ideal, single-axis torque- and rate-limited source, not as an array of physical reaction wheels with their own momentum-storage limits, and not with the full 3-axis orientation model (quaternions) used in a real flight system. Sustained 0.01 N·m for 1800 s would accumulate 18 N·m·s of angular momentum; whether that saturates real wheels depends on the specific hardware chosen, which was not checked.
- **Sensors.** The lidar (laser rangefinder) is sampled at the camera’s rate (5 Hz) rather than its own slower 2 Hz, and its range reading is only used when the camera also detects the target; the brief only requires that the lidar be limited to the camera’s field of view. False alarms occur at 2% *per camera frame* (one spurious detection at a random angle within the field of view; about 180 such events over the 30-minute scenario at 5 Hz), and the track-confirmation rule described above is the only defence against them.
- **Detection curve.** The detection-probability curve is a convenient mathematical approximation of how detection fades with distance, not a model derived from the underlying optics and physics of a real camera.
- **Relative-motion validity.** The equations used to model how the two spacecraft move relative to each other (Clohessy-Wiltshire, described in Section 3) were checked only against their own exact mathematical solution, and are not used to move the chaser in the actual simulation runs (Section 7). Thirty minutes is about a third of one orbit, long enough that a comparison against a more complete, nonlinear orbital model would be needed before trusting positions at that timescale.
- **Metrics not produced.** End-to-end latency from first detection to commanded attitude; estimation error broken down by range; host delta-v; and wheel momentum (only an upper-

bound approximation was computed).

- **Statistics.** 200 runs for the headline result; 80 runs per point in the unpaired sweeps; paired comparisons on extreme settings only, with 100 scenarios each; five comparisons in total, with no test of how parameters might interact with each other. The scenario sampling also excludes very slow, distant approaches that cannot physically complete in time (Section 7), which narrows the range of cases tested.
- **Scope.** The code is a 2D prototype; nothing here is evidence for the fuller 3D design (with a proper orbital estimator and a reaction-wheel/quaternion attitude model) described in Sections 1–6.

## 9 What I’d measure on real hardware to confirm this

A simulation is only as good as the numbers fed into it. This section lists the real, physical measurements I’d want from actual flight hardware and lab testing to check whether the simulation’s assumptions are actually true — because the whole point of a model like this is to guide real engineering decisions, not to exist for its own sake.

- Actual detector dark-current growth versus radiation dose from test data for the specific sensor part, to replace the static detection-probability assumption with a mission-life-aware one.
- Reaction wheel torque ripple and static/dynamic friction from bench characterisation: the 0.01 N·m figure is a limit, and achievable torque accuracy can be worse, particularly near zero wheel speed where friction effects are largest.
- End-to-end onboard latency benchmarked on the actual flight-like compute (not a laptop), including any computer-vision inference step if detection ultimately runs through an onboard vision model on TensorRT/ONNX-class hardware — the prototype assumed a fixed decision latency and found no detectable effect of it, but that reflects the model’s structure rather than reality, and real timing is exactly the kind of number a simulation cannot get right without measurement.
- Thermal-vacuum test data on the actual optics/detector assembly, to check whether on-orbit noise floor matches the ground-calibrated detection curve at all.

## Closing note

I’ve tried to write this the way I’d want to receive it: concrete where I have direct expertise, and explicit about where I don’t. Section 7 reports executed code with real numbers: a denial rate of 13.5% (95% interval 9.4–18.9%), limited mainly by sensing geometry (track initiation at a median true range of 3 m) and by the arming rule and slew rate. One of the brief’s four self-checks, latency injection, does not show the expected effect in this prototype, and I report that as a weakness rather than omit it. The prototype is 2D and simplified, and is not evidence for the full design in Sections 1–6. I’m happy to walk through the code and the limitations on the follow-up call.